



Universidad Mariano
Gálvez de Guatemala.

Proyecto Final – Análisis de Sistemas 2

9989-22-5223- DYLAN ALBERTO ROLDAN PEREZ

9989-21-5704 ANTHONY JAIR ENRIQUE DE LEON DE LEON

9989-23-15707 DULCE CLAUDIA MICHELLE RAMIREZ RAC

9989-12-10925 FELIX FRANCISCO TOMA RODRIGUEZ

Índice

1.	Introducción	5
2.	<i>Contexto del Sistema Legado</i>	6
3.	INGENIERÍA INVERSA DEL SISTEMA ACTUAL	8
3.1	Análisis de la arquitectura existente:	8
3.2	Diagrama del Sistema Heredado:	8
4.	ANÁLISIS DE ALTERNATIVAS ARQUITECTÓNICAS	9
4.1	Alternativas consideradas	9
4.2	<i>Matriz Comparativa de Arquitectura</i>	9
4.3	Arquitectura seleccionada	9
4.4	<i>Justificación técnica de la arquitectura seleccionada</i>	9
5.	Contrato API – OpenAPI 3.0	12
5.1	Descripción de la API	12
5.2	Endpoints	12
5.3	Detalle de endpoints	13
5.3.1	Iniciar sesión	13
5.3.2	Listar contactos	14
5.3.3	Crear contacto	15
5.3.4	Consultar un contacto	16
5.3.5	Actualizar un contacto	17
5.3.6	Desactivar un contacto	18
5.4	DTOs	19
5.5	Códigos HTTP	22
5.6	Seguridad	23
6.	Diseño de Arquitectura C4	24
6.1	Nivel 1 – System Context	24
6.2	Nivel 2 – Container Diagram	26
6.3	Nivel 3 – Component Diagram	26
6.4	Nivel 4 – Code Diagram	26
7.	DISEÑO DE PERSISTENCIA DE DATOS	27
7.1	Tecnología seleccionada	27
7.2	Modelo de datos	28
7.3	Entidades principales	29

8.	<i>Script de la base de datos:</i>	29
9.	<i>Estrategia de Cache</i>	31
9.1	Justificación	31
9.2	Patrón seleccionado	31
9.3	Cache Keys y TTL	32
9.4	Política de invalidación	32
9.5	Flujo de Cache	34
9.6	Cache Hit	34
9.7	Cache Miss	34
9.8	Invalidez/Actualizacion de Cache	34
10.	<i>Matriz de Trazabilidad</i>	34
11.	<i>Validación de la solución</i>	35
11.1	Validación del contrato API	35
11.2	Validación de arquitectura	36
11.3	Consistencia entre API, DTO, modelo de datos y base de datos	37
11.4	Validación de caché	39
12.	<i>CONCLUSIONES</i>	40
13.	<i>REFERENCIAS</i>	41

1. Introducción

El presente documento tiene como propósito realizar el análisis técnico del Módulo CRM heredado, con el fin de identificar su estructura actual, las principales dificultades de mantenimiento y las oportunidades de mejora. El sistema existente fue desarrollado bajo un enfoque monolítico tradicional, en el cual la presentación, las reglas del negocio y el acceso a la base de datos se encuentran estrechamente relacionados, lo que puede generar alto acoplamiento, baja cohesión y dificultades al incorporar nuevas funcionalidades.

Como parte del proceso de ingeniería inversa, se documenta la arquitectura actual mediante un diagrama UML y se identifican los principales elementos de deuda técnica presentes en el sistema. Asimismo, se realiza una comparación entre el monolito heredado y la arquitectura propuesta, evaluando sus ventajas, desventajas y riesgos.

Después del análisis se propone adoptar una arquitectura de **Monolito Modular**, utilizando MVC junto con servicios, repositorios, DTO y middleware para organizar internamente la aplicación y exponer una API REST. Esta decisión se justifica mediante los atributos de calidad de escalabilidad, mantenibilidad, disponibilidad y seguridad, buscando una solución adecuada para el alcance actual del CRM y preparada para evolucionar conforme aumenten sus necesidades.

2. Contexto del Sistema Legado

2.1 Descripción del sistema actual

El sistema actual corresponde a un Módulo CRM desarrollado inicialmente como una aplicación monolítica en PHP, complementada con HTML, CSS y JavaScript, y conectada a una base de datos relacional. Su función principal es administrar usuarios y contactos, permitiendo registrar, consultar, modificar, buscar, activar y desactivar información. Los contactos también pueden clasificarse como generales, clientes, proveedores o empleados.

Se considera un sistema monolítico porque la interfaz, la lógica del negocio y las consultas a la base de datos se encuentran estrechamente relacionadas dentro de los mismos archivos o módulos. Al no utilizar una arquitectura como MVC, algunas páginas se encargan simultáneamente de recibir solicitudes, validar datos, ejecutar consultas y generar el contenido mostrado al usuario.

Entre sus principales puntos de dolor se encuentra el alto acoplamiento, debido a que un cambio en la base de datos o en una regla del negocio puede requerir modificaciones en diferentes partes del sistema. También presenta baja cohesión, duplicación de código y dificultad para realizar pruebas independientes. Conforme se agregan nuevas funciones, como inventarios, costos o automatizaciones, aumenta la complejidad del mantenimiento. Además, las consultas ejecutadas directamente desde distintos archivos pueden convertirse en cuellos de botella y dificultar el crecimiento y la escalabilidad del sistema.

2.2 Tecnologías Actuales

Elemento	Tecnología
Lenguaje	PHP
Framework	Ninguno
Base de datos	MySQL
Fronted	HTML, CSS, JavaScript
Infraestructura	Servidor unico

2.3 Puntos de dolor identificados

Problema	Causa	consecuencia	evidencia
acoplamiento	Logica de negocio, el acceso a datos y la presentacion estan en el mismo archivo	Un cambio en la base de datos obliga a modificar varios archivos a la vez	Archivos como contactos.php mexcaln SQL, validacion y HTML
Cuello de botella	Consultas SQL ejecutada directamente desde cada pagina	Bajo rendimiento cuando crece el volumen de contactos	Consultas repetidas de SELECT*FROM contactos en multiples visitas

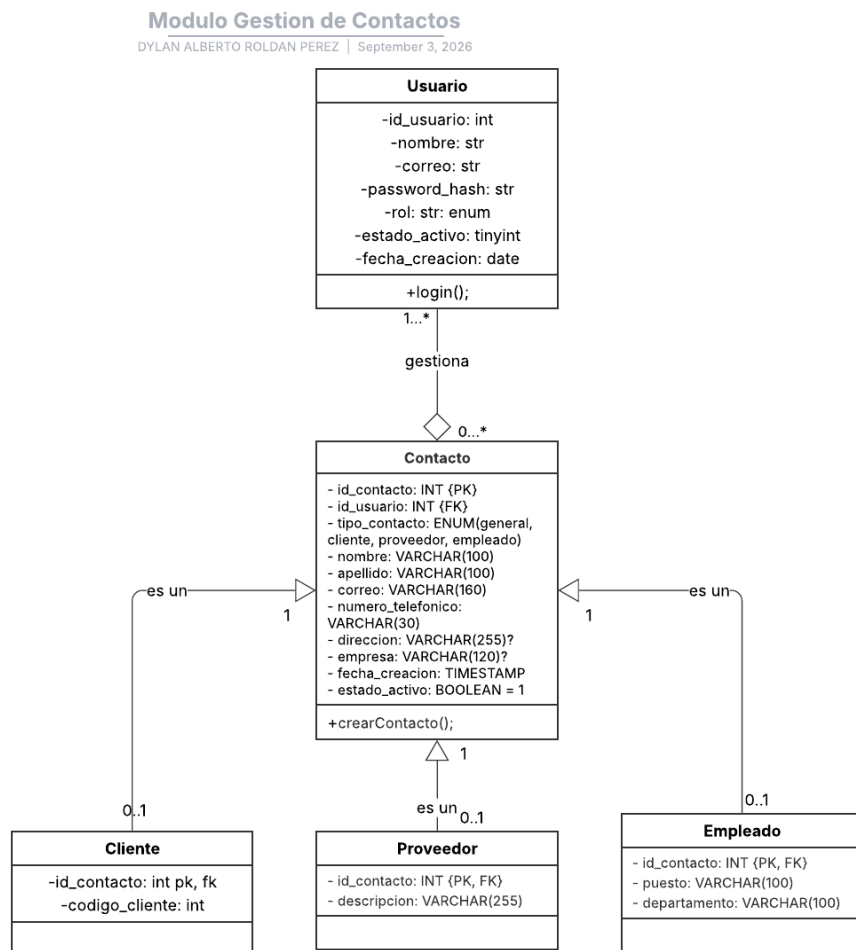
	sin capa de repositorio		
Problemas de escalabilidad	Arquitectura monolítica sin separación de responsabilidades	No se puede escalar un módulo sin escalar todo el sistema	No existen módulos independientes desplegables
Problemas de mantenibilidad	Duplica de código y ausencia de patrón MVC	Corregir un error puede requerir cambios en varios archivos	Validaciones de formulario repetidas en distintos archivos
Problemas de disponibilidad	Un solo punto de falla	Un error no controlado puede caer toda la aplicación	No hay manejo centralizado de errores ni monitoreo de salud
Riesgo de seguridad	Falta de capa de autenticación centralizada: posibles consultas SQL no parametrizadas	Riesgo de inyección SQL y acceso no autorizado a datos	Contraseñas gestionadas sin evidencia de middleware de autorización
Dependencias entre componentes	Cliente, proveedor y empleado dependen directamente de la escritura interna de contacto	Un cambio en contacto puede romper las tres subclases a la vez	Relación de herencia rígida en el diagrama de clases actual

3. INGENIERÍA INVERSA DEL SISTEMA ACTUAL

3.1 Análisis de la arquitectura existente:

Esta es monolítica, tiene un alto acoplamiento ya que varios archivos contienen lógica y sentencias sql, lo cual al hacer un cambio afectaría varias partes del código.

3.2 Diagrama del Sistema Heredado:



4. ANÁLISIS DE ALTERNATIVAS ARQUITECTÓNICAS

4.1 Alternativas consideradas

Solamente se considero la monolito modular, microservicios implica mucha agregaría un grado de complejidad innecesaria al proyecto.

4.2 Matriz Comparativa de Arquitectura

Evaluación	Sistema actual: monolito heredado	Arquitectura propuesta: monolito modular
Pros	Un solo despliegue; infraestructura sencilla; bajo costo inicial; acceso directo y rápido a la base de datos; funcionamiento suficiente para operaciones básicas del CRM.	Mantiene un solo despliegue; divide el sistema en módulos organizados; reduce el acoplamiento; mejora la mantenibilidad; facilita las pruebas; centraliza la seguridad y el acceso a datos; permite integrar una API y Redis; puede evolucionar gradualmente.
Contras	Alto acoplamiento; baja cohesión; mezcla de presentación, lógica y consultas SQL; posible duplicación de código; dificultad para realizar pruebas; mantenimiento complicado; poca flexibilidad para agregar nuevas funciones.	Requiere mayor planificación inicial; los módulos comparten el mismo proceso y despliegue; no permite escalar cada módulo de manera independiente; necesita disciplina para respetar los límites entre módulos.
Riesgos	Una modificación puede afectar varias funciones; una falla puede detener toda la aplicación; aumento progresivo de la deuda técnica; mayor dificultad para corregir vulnerabilidades; posibles cuellos de botella en las consultas directas.	Puede regresar a un monolito desorganizado si los módulos se comunican directamente; una falla crítica todavía podría afectar toda la aplicación; crecimiento excesivo de la base de datos compartida; dependencia incorrecta entre módulos si no se aplican reglas arquitectónicas.

4.3 Arquitectura seleccionada

Arquitectura propuesta: Monolítica Modular

4.4 Justificación técnica de la arquitectura seleccionada

Para exponer la nueva API del Módulo CRM se seleccionó una arquitectura de **Monolito Modular**, implementada internamente mediante MVC y capas de servicios, repositorios, DTO y middleware. Esta decisión permite mantener una sola aplicación y un único despliegue, pero separando las funcionalidades en módulos como autenticación, usuarios, contactos, clientes, proveedores y empleados.

Escalabilidad

Aunque el Monolito Modular se despliega como una sola unidad y no permite escalar cada módulo de forma independiente, ofrece una capacidad de crecimiento suficiente para el alcance actual del CRM. La API podrá ejecutar varias instancias de la aplicación detrás de un balanceador de carga si aumenta la cantidad de solicitudes.

También se utilizarán paginación, índices en la base de datos y Redis mediante el patrón Cache-Aside para reducir consultas repetidas. La autenticación mediante JWT contribuirá a que las instancias de la API no dependan de sesiones almacenadas localmente. Además, la separación modular facilitará extraer un módulo y convertirlo en microservicio si en el futuro presenta una carga considerablemente mayor.

Mantenibilidad

La arquitectura propuesta mejora la mantenibilidad porque establece responsabilidades específicas para cada elemento. Las rutas definirán los endpoints; los controladores recibirán las solicitudes; los servicios contendrán las reglas del negocio; los repositorios administrarán el acceso a los datos; y los DTO establecerán la información que la API puede recibir y devolver.

El flujo general será:

Ruta → Controlador → Servicio → Repositorio → Modelo → Base de datos

Esta separación reduce el acoplamiento y aumenta la cohesión. Por ejemplo, una modificación en las reglas para registrar un cliente podrá realizarse dentro del módulo correspondiente sin alterar innecesariamente la autenticación o la gestión de proveedores. También facilitará la creación de pruebas unitarias y de integración, evitando controladores con demasiadas responsabilidades.

Disponibilidad

Al tratarse de una sola aplicación, una falla crítica podría afectar toda la API. Para reducir este riesgo se propone utilizar manejo centralizado de excepciones, registros de errores, verificaciones de salud, copias de seguridad, pruebas antes de cada despliegue y un procedimiento de reversión hacia una versión estable.

En caso de crecimiento, podrán ejecutarse varias instancias de la aplicación para evitar que el servicio dependa de un único proceso. Redis funcionará únicamente como almacenamiento temporal y MySQL continuará siendo la fuente oficial de información. Por lo tanto, si Redis deja de estar disponible, la API podrá consultar directamente la base de datos, posiblemente

con un menor rendimiento, pero sin perder información ni interrumpir las operaciones principales.

Seguridad

La seguridad se centralizará en la API mediante autenticación con JWT y el esquema bearerAuth. Los endpoints protegidos utilizarán middleware de autenticación y autorización basada en roles, diferenciando las acciones permitidas para usuarios estándar y administradores.

Las contraseñas se almacenarán mediante funciones de hash seguras y nunca serán expuestas en las respuestas. Los DTO evitarán devolver campos internos como password_hash. También se aplicará validación de datos, consultas parametrizadas mediante el ORM, control de acceso, limitación de solicitudes y comunicación por HTTPS. Las credenciales y claves privadas deberán almacenarse en variables de entorno y no directamente en el código fuente.

Conclusión

El Monolito Modular representa la alternativa más adecuada para el Módulo CRM porque ofrece un equilibrio entre escalabilidad, mantenibilidad, disponibilidad, seguridad y complejidad. Corrige los principales problemas del monolito heredado, como el alto acoplamiento y la mezcla de responsabilidades, sin introducir los costos operativos de una arquitectura de microservicios.

MVC organizará internamente la aplicación, mientras que los módulos, servicios, repositorios, DTO y middleware permitirán exponer una API ordenada, segura y preparada para crecer. De esta manera, se conserva la sencillez de administrar una sola aplicación y se establece una base arquitectónica que podrá evolucionar si las necesidades del sistema aumentan.

5. Contrato API – OpenAPI 3.0

5.1 Descripción de la API

Nombre: API Módulo CRM - Gestión de Contactos.

Propósito: Exponer las operaciones de consulta, registro, actualización y desactivación de contactos mediante solicitudes HTTP. La API permite gestionar contactos generales, clientes, proveedores y empleados, y obtener un token para acceder a las operaciones protegidas.

Versión: 1.0.0. El prefijo de las rutas es /api/v1. La especificación utiliza OpenAPI 3.0.3, que corresponde a la versión del formato del contrato.

URL base de desarrollo: http://127.0.0.1:8000/api/v1.

Recursos principales: Contactos, con datos generales y detalles según su clasificación; autenticación, para validar credenciales y emitir el token. El inicio de sesión también devuelve un resumen del usuario autenticado.

Consumidores previstos: Aplicaciones web o móviles e integraciones externas que necesiten gestionar contactos. Durante el desarrollo, las operaciones pueden consultarse mediante clientes de pruebas como Postman o Swagger UI. El acceso a los contactos requiere credenciales válidas y un token.

Formato de intercambio: JSON, mediante el tipo de contenido application/json.

5.2 Endpoints

Las rutas de la tabla se agregan a la URL base. Cada combinación de método y ruta representa una operación.

Método	Endpoint	Operación	Autenticación
POST	/auth/login	Iniciar sesión y obtener el token	Sin token previo
GET	/contactos	Listar, buscar y filtrar contactos	Bearer JWT
POST	/contactos	Crear un contacto	Bearer JWT
GET	/contactos/{id}	Consultar un contacto	Bearer JWT
PUT	/contactos/{id}	Actualizar un contacto	Bearer JWT
DELETE	/contactos/{id}	Desactivar un contacto	Bearer JWT

DELETE realiza una desactivación lógica: conserva el registro y cambia estado_activo a false.

5.3 Detalle de endpoints

Las solicitudes con cuerpo utilizan Content-Type: application/json. Los endpoints protegidos requieren el encabezado Authorization: Bearer <token>. Los ejemplos muestran datos ilustrativos; el token representado debe sustituirse por uno obtenido al iniciar sesión.

5.3.1 Iniciar sesión

Método HTTP: POST.

URL: http://127.0.0.1:8000/api/v1/auth/login.

Descripción: Valida el correo y la contraseña del usuario y devuelve un token Bearer junto con un resumen de su cuenta.

Parámetros: No utiliza parámetros de ruta ni de consulta. Las credenciales se envían en el cuerpo.

Request Body: Obligatorio. correo debe tener formato de correo electrónico y un máximo de 160 caracteres; password debe tener al menos 8 caracteres. Ambos campos son obligatorios.

```
{  
  "correo": "administrador@crm.com",  
  "password": "Clave1234"  
}
```

DTO utilizado: LoginRequest en la solicitud y LoginResponse en la respuesta. Los datos de la cuenta utilizan UsuarioResumen.

Respuesta: 200 OK, con la siguiente estructura:

```
{  
  "success": true,  
  "message": "Inicio de sesión correcto.",  
  "data": {  
    "token": "eyJhbGciOiJIUzI1NiJ9.ejemplo.firma",  
    "token_type": "Bearer",  
    "usuario": {  
      "id_usuario": 1,
```

```

"nombre": "Administrador CRM",
"correo": "administrador@crm.com",
"rol": "administrador",
"estado_activo": true
}
}
}

```

Códigos HTTP: 200 inicio de sesión correcto; 400 solicitud incorrecta; 401 credenciales inválidas o usuario inactivo; 500 error interno del servidor.

Seguridad: No requiere un token previo. La autenticación se realiza con las credenciales enviadas. El token devuelto permite realizar solicitudes a los endpoints protegidos.

5.3.2 Listar contactos

Método HTTP: GET.

URL: http://127.0.0.1:8000/api/v1/contactos.

Descripción: Devuelve una lista paginada de los contactos visibles para el usuario autenticado. Permite buscar y filtrar los resultados.

Parámetros: Todos se envían en la cadena de consulta y son opcionales.

Parámetro	Tipo	Uso y valores permitidos
buscar	string	Busca por nombre, apellido, correo, teléfono o empresa.
tipo	string	general, cliente, proveedor o empleado.
estado	string	activo o inactivo.
page	integer	Página solicitada. Mínimo 1; valor predeterminado 1.
per_page	integer	Registros por página. De 1 a 100; valor predeterminado 10.

Ejemplo de consulta:

/contactos?buscar=Carlos&tipo=cliente&estado=activo&page=1&per_page=10.

Request Body: No utiliza cuerpo de solicitud.

DTO utilizado: ContactoListResponse, que contiene una lista de objetos Contacto y un objeto PaginationMeta.

Respuesta: 200 OK. Incluye success, message, data y meta. Cada elemento de data sigue el esquema del contacto mostrado en el apartado 5.3.4. Un ejemplo válido de una página sin resultados es:

```
{
  "success": true,
  "message": "Contactos consultados correctamente.",
  "data": [],
  "meta": {
    "pagina_actual": 1,
    "ultima_pagina": 1,
    "por_pagina": 10,
    "total": 0
  }
}
```

Códigos HTTP: 200 consulta correcta; 401 token ausente o inválido; 500 error interno del servidor.

Seguridad: Requiere Bearer JWT. El listado queda limitado a los contactos que el usuario autenticado puede visualizar.

5.3.3 Crear contacto

Método HTTP: POST.

URL: http://127.0.0.1:8000/api/v1/contactos.

Descripción: Registra un contacto general, cliente, proveedor o empleado con sus datos generales y los campos específicos de su tipo.

Parámetros: No utiliza parámetros de ruta ni de consulta.

Request Body: Obligatorio. Requiere tipo_contacto, nombre, apellido y numero_telefonico. También requiere codigo_cliente para clientes, descripcion para proveedores, y puesto y departamento para empleados, según las reglas descritas en el contrato. Ejemplo de registro de un cliente:

```
{
  "tipo_contacto": "cliente",
```

```
"nombre": "Carlos",
"apellido": "López",
"correo": "carlos@example.com",
"numero_telefonico": "+502 5555-5555",
"direccion": "Ciudad de Guatemala",
"empresa": "Comercial López",
"codigo_cliente": 1001
}
```

DTO utilizado: ContactoCreateRequest en la solicitud y ContactoResponse en la respuesta. Las restricciones de cada campo se detallan en el apartado 5.4.

Respuesta: 201 Created. Devuelve success, message y data, donde data contiene el contacto creado con su identificador, el identificador del usuario asociado, su estado y sus fechas. Su estructura coincide con el ejemplo de ContactoResponse del apartado 5.3.4.

Códigos HTTP: 201 contacto creado; 400 solicitud incorrecta; 401 token ausente o inválido; 422 error de validación; 500 error interno del servidor.

Seguridad: Requiere Bearer JWT. id_contacto, id_usuario, estado_activo y las fechas no forman parte del DTO de creación; aparecen en la respuesta.

5.3.4 Consultar un contacto

Método HTTP: GET.

URL: http://127.0.0.1:8000/api/v1/contactos/{id}.

Descripción: Devuelve la información general y los detalles específicos de un contacto.

Parámetros: id, obligatorio en la ruta; entero de 64 bits con valor mínimo de 1. Ejemplo: /contactos/15.

Request Body: No utiliza cuerpo de solicitud.

DTO utilizado: ContactoResponse, cuyo campo data utiliza el esquema Contacto.

Respuesta: 200 OK. Ejemplo de un contacto de tipo cliente:

```
{
"success": true,
"message": "Operación realizada correctamente.",
"data": {
```

```
"id_contacto": 15,  
"id_usuario": 1,  
"tipo_contacto": "cliente",  
"nombre": "Carlos",  
"apellido": "López",  
"correo": "carlos@example.com",  
"numero_telefonico": "+502 5555-5555",  
"direccion": "Ciudad de Guatemala",  
"empresa": "Comercial López",  
"estado_activo": true,  
"fecha_creacion": "2026-09-21T06:00:00Z",  
"fecha_actualizacion": null,  
"cliente": {"codigo_cliente": 1001},  
"proveedor": null,  
"empleado": null  
}  
}
```

Códigos HTTP: 200 contacto encontrado; 401 token ausente o inválido; 403 sin permiso para consultar el contacto; 404 contacto no encontrado; 500 error interno del servidor.

Seguridad: Requiere Bearer JWT y autorización para consultar el contacto identificado en la ruta.

5.3.5 Actualizar un contacto

Método HTTP: PUT.

URL: `http://127.0.0.1:8000/api/v1/contactos/{id}`.

Descripción: Modifica los datos generales y la clasificación de un contacto existente.

Parámetros: id, obligatorio en la ruta; entero de 64 bits con valor mínimo de 1. Ejemplo: `/contactos/15`.

Request Body: Obligatorio. `ContactoUpdateRequest` reutiliza `ContactoCreateRequest` mediante `allOf`, por lo que mantiene sus campos obligatorios y restricciones. Deben enviarse

tipo_contacto, nombre, apellido y numero_telefonico, además de los campos exigidos por el tipo de contacto. Ejemplo:

```
{
  "tipo_contacto": "cliente",
  "nombre": "Carlos",
  "apellido": "López",
  "correo": "carlos@example.com",
  "numero_telefonico": "+502 5555-7777",
  "direccion": "Zona 1, Ciudad de Guatemala",
  "empresa": "Comercial López",
  "codigo_cliente": 1001
}
```

DTO utilizado: ContactoUpdateRequest en la solicitud y ContactoResponse en la respuesta.

Respuesta: 200 OK. Devuelve success, message y el contacto actualizado dentro de data, con la misma estructura del apartado 5.3.4.

Códigos HTTP: 200 contacto actualizado; 400 solicitud incorrecta; 401 token ausente o inválido; 403 sin permiso para actualizar; 404 contacto no encontrado; 422 error de validación; 500 error interno del servidor.

Seguridad: Requiere Bearer JWT y autorización para modificar el contacto indicado. El contrato documenta PUT; no define una operación PATCH.

5.3.6 Desactivar un contacto

Método HTTP: DELETE.

URL: http://127.0.0.1:8000/api/v1/contactos/{id}.

Descripción: Desactiva lógicamente un contacto, conservando el registro en la base de datos.

Parámetros: id, obligatorio en la ruta; entero de 64 bits con valor mínimo de 1. Ejemplo: /contactos/15.

Request Body: No utiliza cuerpo de solicitud.

DTO utilizado: ContactoEstadoResponse en la respuesta.

Respuesta: 200 OK, con el identificador del contacto y su estado inactivo:

```
{
  "success": true,
  "message": "Contacto desactivado correctamente.",
  "data": {
    "id_contacto": 15,
    "estado_activo": false
  }
}
```

Códigos HTTP: 200 contacto desactivado; 401 token ausente o inválido; 403 sin permiso para desactivar; 404 contacto no encontrado; 500 error interno del servidor.

Seguridad: Requiere Bearer JWT y autorización para desactivar el contacto indicado.

5.4 DTOs

Un DTO, u objeto de transferencia de datos, define la estructura de la información que una solicitud envía o que una respuesta devuelve. En el contrato, estas estructuras se encuentran en components/schemas. Los esquemas pueden orientar la validación y la serialización en Laravel, sin exigir que cada uno corresponda a una clase PHP con el mismo nombre.

DTO	Tipo	Descripción
LoginRequest	Request	Credenciales obligatorias: correo y password.
ContactoCreateRequest	Request	Datos generales y específicos para crear un contacto.
ContactoUpdateRequest	Request	Reutiliza las propiedades y requisitos de ContactoCreateRequest.
LoginResponse	Response	success, message y data con token, token_type y usuario.
UsuarioResumen	Response	id_usuario, nombre, correo, rol y estado_activo; todos obligatorios.
ContactoResponse	Response	success, message y data con un objeto Contacto.

ContactoListResponse	Response	success, message, data como lista de contactos y meta con la paginación.
ContactoEstadoResponse	Response	success, message y data con id_contacto y estado_activo.
Contacto	Response	Datos generales del contacto y detalles de cliente, proveedor o empleado.
ClienteDetalle	Response	codigo_cliente, de tipo entero.
ProveedorDetalle	Response	descripcion, de tipo string.
EmpleadoDetalle	Response	puesto y departamento, de tipo string.
PaginationMeta	Response	pagina_actual, ultima_pagina, por_pagina y total; enteros obligatorios.
ErrorResponse	Response	success y message obligatorios; errors opcional y admite null.
ValidationErrorResponse	Response	success, message y errors obligatorios; errors contiene listas de mensajes por campo.

Campos de creación y actualización

La misma definición se aplica a ContactoCreateRequest y ContactoUpdateRequest. Los campos opcionales de esta tabla admiten null en el esquema.

Campo	Tipo	Obligatorio	Restricciones
tipo_contacto	string	Sí	general, cliente, proveedor o empleado.
nombre	string	Sí	Entre 2 y 100 caracteres.
apellido	string	Sí	Entre 2 y 100 caracteres.
numero_telefonico	string	Sí	Entre 8 y 30 caracteres.
correo	string	No	Formato email; máximo 160 caracteres.
direccion	string	No	Máximo 255 caracteres.

empresa	string	No	Máximo 120 caracteres.
codigo_cliente	integer	Si es cliente	Mínimo 1.
descripcion	string	Si es proveedor	Máximo 255 caracteres.
puesto	string	Si es empleado	Máximo 100 caracteres.
departamento	string	Si es empleado	Máximo 100 caracteres.

Las obligaciones que dependen del tipo de contacto están expresadas en las descripciones de los campos. El esquema marca esos campos como nullable y no formaliza la condición mediante esquemas alternativos; la validación del servidor debe aplicar la regla de negocio correspondiente.

Campos del contacto devuelto

Contacto requiere id_contacto, id_usuario, tipo_contacto, nombre, apellido, numero_telefonico, estado_activo y fecha_creacion. Los identificadores son enteros; el estado es booleano; la fecha utiliza el formato date-time.

También define correo, direccion, empresa y fecha_actualizacion como propiedades opcionales que admiten null. fecha_actualizacion utiliza el formato date-time cuando tiene un valor. Los objetos cliente, proveedor y empleado son opcionales y admiten null; permiten incluir los datos específicos de la clasificación.

En la solicitud, los datos específicos se envían como campos directos, por ejemplo codigo_cliente. En la respuesta, se agrupan en su objeto correspondiente, por ejemplo cliente.codigo_cliente.

Estructuras de error

Los errores generales utilizan ErrorResponse. Ejemplo de un contacto no encontrado:

```
{
  "success": false,
  "message": "Contacto no encontrado.",
  "errors": null
}
```

Los errores de validación utilizan `ValidationErrorResponse`, donde cada clave de errors identifica un campo y contiene una lista de mensajes. Ejemplo:

```
{  
  "success": false,  
  "message": "Los datos proporcionados no son válidos.",  
  "errors": {  
    "nombre": ["El nombre debe contener al menos 2 caracteres."]  }  
}
```

5.5 Códigos HTTP

Los códigos permiten que las aplicaciones consumidoras identifiquen si una operación fue correcta o qué clase de error ocurrió. Cada endpoint utiliza únicamente las respuestas especificadas para él en el apartado 5.3.

Código	Uso en la API
200 OK	Inicio de sesión correcto, consulta de contactos, actualización o desactivación realizada correctamente.
201 Created	Creación correcta de un contacto mediante POST /contactos.
400 Bad Request	Solicitud incorrecta en el inicio de sesión, la creación o la actualización de un contacto.
401 Unauthorized	Credenciales inválidas o usuario inactivo al iniciar sesión; token ausente o inválido en los endpoints protegidos.
403 Forbidden	Usuario autenticado sin permiso para consultar, actualizar o desactivar un contacto específico.
404 Not Found	No se encuentra el contacto indicado en las operaciones GET, PUT o DELETE /contactos/{id}.
422 Unprocessable Entity	Los datos no cumplen las validaciones al crear o actualizar un contacto. Devuelve <code>ValidationErrorResponse</code> .

500 Internal Server Error	Error interno del servidor al procesar cualquiera de las operaciones.
---------------------------	---

400 indica una solicitud incorrecta; 422 identifica los errores de validación de datos que el contrato distingue en la creación y actualización. 401 corresponde a un problema de autenticación y 403 a la falta de permisos sobre el contacto.

5.6 Seguridad

Bearer Authentication y JWT

El contrato define el esquema de seguridad bearerAuth, de tipo HTTP, con el mecanismo bearer y el formato de token JWT. Bearer describe cómo se presenta el token en la solicitud; JWT identifica el formato previsto para ese token.

El consumidor envía correo y password a POST /auth/login. Cuando el acceso es válido, obtiene el token en data.token y el tipo en data.token_type. En las siguientes solicitudes a contactos debe enviar:

Authorization: Bearer <token>

Accept: application/json

Cuando la solicitud incluye un cuerpo JSON, también se utiliza Content-Type: application/json. Un token ausente o inválido produce una respuesta 401 Unauthorized en las operaciones protegidas.

Autorización

La autenticación identifica al usuario. La autorización determina si puede realizar una operación sobre un contacto. El listado devuelve únicamente los contactos visibles para la cuenta autenticada; la consulta individual, la actualización y la desactivación contemplan 403 Forbidden cuando faltan permisos.

El esquema UsuarioResumen reconoce los roles estandar y administrador. La matriz concreta de permisos por rol y las reglas de acceso a contactos deben definirse en la implementación, porque el contrato no las detalla. El tipo de contacto empleado es una clasificación del contacto, no uno de los roles del usuario.

Endpoints protegidos

Las cinco operaciones de contactos requieren bearerAuth: GET /contactos, POST /contactos, GET /contactos/{id}, PUT /contactos/{id} y DELETE /contactos/{id}. POST /auth/login permite obtener el token y no exige uno previo.

Alcance de la configuración

El contrato no fija el tiempo de expiración, el algoritmo de firma ni un endpoint de renovación o cierre de sesión. Estos aspectos deben establecerse en la configuración de autenticación del sistema. La URL HTTP declarada corresponde al entorno local; para publicar la API se recomienda HTTPS, que protege las credenciales y el token durante su transmisión.

Evidencias:

[Especificacion API](#)

[Swagger UI](#)

6. Diseño de Arquitectura C4

6.1 [Nivel 1 – System Context](#)

Usuarios

En el diagrama existen dos tipos de usuarios:

Usuario Estándar: representa al empleado que utiliza el Módulo CRM para trabajar con la información de los contactos. Puede realizar operaciones como consultar, registrar y actualizar contactos, incluyendo clientes, proveedores y empleados.

Administrador: representa al usuario con mayores privilegios dentro del sistema. Además de trabajar con contactos, tiene permisos para administrar usuarios y realizar tareas relacionadas con la configuración del sistema.

La diferencia principal es que el usuario estándar se concentra en las operaciones habituales del CRM, mientras que el administrador posee funciones adicionales de gestión y control.

Sistema principal

El sistema principal es el **Sistema Módulo CRM**.

Su función es centralizar la gestión de contactos de la organización y permitir operaciones como:

- Registrar contactos.
- Consultar información.
- Modificar datos.

- Buscar contactos.
- Activar o desactivar registros.
- Gestionar clientes, proveedores y empleados.

El sistema también expone estas funcionalidades mediante una **API REST**, permitiendo que las operaciones se realicen de forma estructurada entre la aplicación y el backend.

En un diagrama C4 Nivel 1 no se muestran todavía detalles internos como controllers, services, base de datos o Redis. Todo eso se representa simplemente como un único sistema: **Sistema Módulo CRM**.

Sistemas externos

En el diagrama aparece como sistema externo el **Servicio de Correo**.

Este servicio no forma parte directamente del Módulo CRM, sino que es utilizado por este para enviar notificaciones, por ejemplo:

- Confirmaciones de registro.
- Alertas.
- Avisos relacionados con operaciones del sistema.

La comunicación puede realizarse mediante **SMTP o una API de correo**.

Se considera externo porque el CRM lo consume, pero su funcionamiento interno no es responsabilidad del sistema desarrollado.

Relaciones

Las relaciones indican **cómo interactúan los usuarios y sistemas entre sí**.

Usuario Estándar → Sistema Módulo CRM: el usuario consulta, registra y actualiza contactos. La comunicación se realiza mediante **HTTPS**, permitiendo que la información viaje de forma cifrada entre el cliente y el sistema.

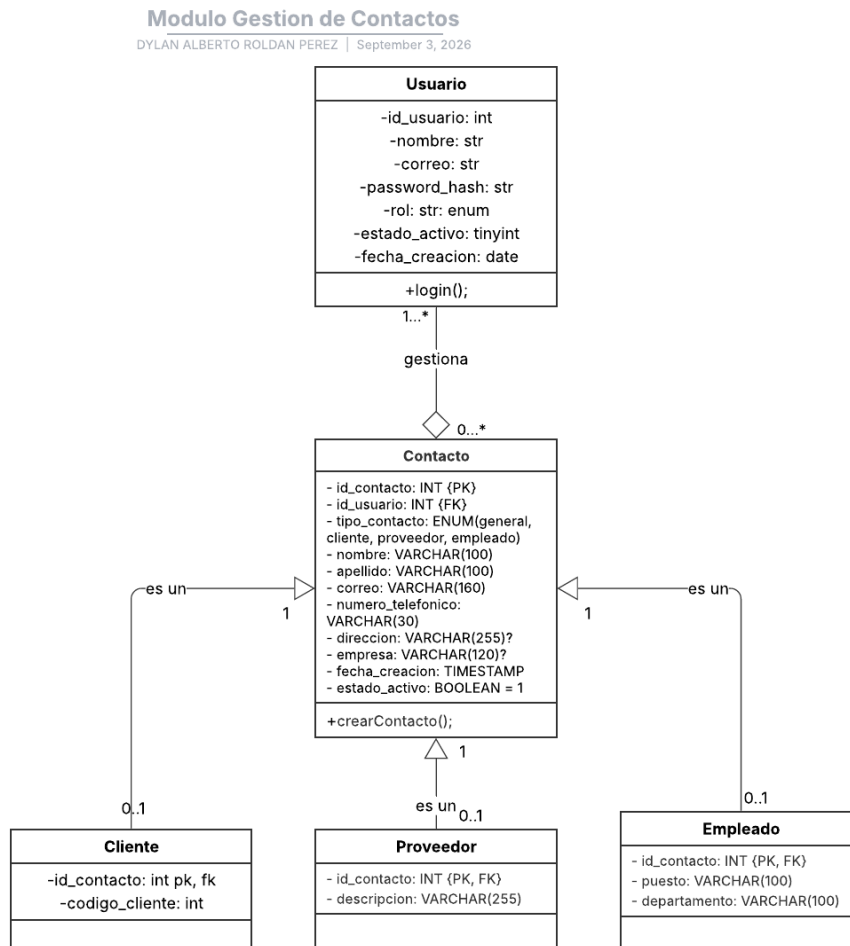
Administrador → Sistema Módulo CRM: el administrador utiliza el sistema para administrar usuarios y contactos. Al igual que el usuario estándar, la comunicación se realiza mediante **HTTPS**.

Sistema Módulo CRM → Servicio de Correo: el CRM utiliza el servicio externo para enviar notificaciones. Esta comunicación puede realizarse mediante **SMTP o una API**.

6.2 Nivel 2 – Container Diagram

6.3 Nivel 3 – Component Diagram

6.4 Nivel 4 – Code Diagram



7. DISEÑO DE PERSISTENCIA DE DATOS

7.1 Tecnología seleccionada

Motor de base de datos

El motor de base de datos seleccionado para el sistema es **MySQL**.

MySQL se utiliza para almacenar la información principal del Módulo CRM, como:

- Usuarios.
- Contactos.
- Clientes.
- Proveedores.
- Empleados.
- Roles y datos relacionados con la gestión del sistema.

Tipo de base de datos

MySQL es una base de datos **relacional**.

Esto significa que la información se organiza en tablas que pueden relacionarse entre sí mediante claves primarias y foráneas. Por ejemplo, un contacto puede estar asociado con información específica de un cliente, proveedor o empleado.

Justificación de la selección

Se seleccionó **MySQL** porque el Módulo CRM maneja información estructurada y con relaciones bien definidas entre sus entidades. El uso de una base de datos relacional facilita mantener la integridad y consistencia de los datos.

Además, MySQL ofrece características adecuadas para el proyecto:

- **Integridad de datos:** permite utilizar claves primarias, claves foráneas y restricciones para evitar registros inconsistentes.
- **Consultas estructuradas:** mediante SQL es posible realizar búsquedas, filtros, actualizaciones y relaciones entre tablas de forma eficiente.
- **Compatibilidad:** puede integrarse fácilmente con el backend y con herramientas ORM utilizadas en aplicaciones web.
- **Transacciones:** permite ejecutar operaciones de manera segura, manteniendo la consistencia de los datos ante errores.

- **Escalabilidad:** soporta el crecimiento de registros y usuarios esperado para un sistema CRM.
- **Mantenibilidad:** al trabajar con tablas y relaciones claramente definidas, facilita futuras modificaciones del sistema.
- **Madurez y soporte:** MySQL es ampliamente utilizado en aplicaciones web y dispone de abundante documentación y herramientas de administración.

En el proyecto también aparece **Redis**, pero es importante diferenciarlo: **Redis no reemplaza a MySQL**. Redis es una base de datos NoSQL en memoria utilizada como **caché** para guardar temporalmente resultados de consultas frecuentes y mejorar el rendimiento. La información permanente del CRM continúa almacenándose en MySQL.

7.2 Modelo de datos

El módulo CRM utiliza un **modelo de datos relacional**, organizado en tablas; por ello, no utiliza colecciones NoSQL.

Entidad	Descripción
Usuarios	Almacena las cuentas de acceso, el correo, la contraseña protegida, el rol y el estado.
Contactos	Guarda los datos generales: nombre, apellido, correo, teléfono, dirección, empresa y tipo de contacto.
Clientes	Complementa los datos del contacto con un código único de cliente.
Proveedores	Añade una descripción específica del proveedor.
Empleados	Registra el puesto y departamento del contacto clasificado como empleado.

Relaciones principales:

- **Usuario–Contactos (1):** un usuario puede tener varios contactos registrados; cada contacto pertenece a un usuario mediante `id_usuario`.
- **Contacto–Cliente, Proveedor o Empleado (1:0..1):** un contacto puede tener un registro complementario en la tabla correspondiente a su tipo. Estas tablas utilizan `id_contacto` como clave primaria y foránea.

Esta estructura centraliza los datos comunes en **Contactos** y separa los atributos específicos para evitar duplicación.

7.3 Entidades principales

Entidad	Responsabilidad	Atributos principales
Usuario	Gestionar el acceso al sistema y asociar los contactos registrados.	id_usuario, nombre, correo, password_hash, rol, estado_activo.
Contacto	Centralizar la información general de los contactos.	id_contacto, id_usuario, tipo_contacto, nombre, apellido, correo, numero_telefonico, direccion, empresa, estado_activo.
Cliente	Almacenar la información específica de los contactos de tipo cliente.	id_contacto, codigo_cliente.
Proveedor	Almacenar la información específica de los contactos de tipo proveedor.	id_contacto, descripcion.
Empleado	Almacenar la información laboral de los contactos de tipo empleado.	id_contacto, puesto, departamento.

8. Script de la base de datos:

Elemento	Descripción
Creación de tablas	Crea la base modolocrmLaravel y las tablas principales usuarios, contactos, clientes, proveedores y empleados. Incluye tablas auxiliares de Laravel para sesiones, caché, tareas y migraciones. Al utilizar MySQL, se trabaja con tablas relacionales.
Claves primarias	id_usuario identifica a cada usuario e id_contacto identifica a cada contacto. Las tablas de clientes, proveedores y empleados también utilizan id_contacto como clave primaria.

Elemento	Descripción
Claves foráneas	Relacionan cada contacto con su usuario responsable y cada cliente, proveedor o empleado con su contacto. Impiden eliminar usuarios con contactos asociados y eliminan los datos del subtipo si se elimina físicamente el contacto.
Restricciones	Utiliza NOT NULL para campos obligatorios, UNIQUE para evitar duplicados en correos, teléfonos y códigos de cliente, y ENUM para limitar roles y tipos de contacto. Las validaciones de formato se realizan en Laravel.
Índices	Incluye índices en usuario responsable, tipo de contacto, estado y la combinación nombre-apellido, además de los índices de claves primarias y valores únicos.
Datos de prueba — seeds—	Inserta dos usuarios y cinco contactos ficticios, incluyendo un cliente, un proveedor, un empleado y contactos generales. Contiene registros activos e inactivos y contraseñas almacenadas mediante bcrypt.

[SCRIPT SQL](#)

9. Estrategia de Cache

9.1 Justificación

La estrategia de caché del Módulo CRM estará orientada a almacenar temporalmente información de consulta frecuente, principalmente el detalle de contactos, los listados filtrados y las estadísticas mostradas en el dashboard.

Entre la información que se almacenará en caché se encuentran los datos de contactos consultados por identificador, los resultados de listados asociados a un usuario y sus filtros, y los conteos utilizados en el panel principal, como contactos totales, activos, inactivos, clientes, proveedores, empleados y contactos generales.

El uso de caché permitirá reducir la cantidad de consultas realizadas directamente a MySQL. Esto resulta especialmente útil en operaciones `GET` que pueden ejecutarse repetidamente sin que la información cambie entre una solicitud y otra.

El problema que se pretende resolver es la ejecución innecesaria de consultas repetidas sobre la base de datos. Al utilizar Redis como almacenamiento temporal, algunas solicitudes podrán responderse directamente desde memoria, reduciendo el tiempo de respuesta de la API y la carga sobre MySQL.

No se almacenarán en caché contraseñas, hashes de contraseñas, tokens completos de autenticación ni información sensible relacionada con las credenciales de los usuarios.

9.2 Patrón seleccionado

Para el Módulo CRM se utilizará el patrón **Cache-Aside**.

En este patrón, la aplicación Laravel es responsable de consultar primero Redis antes de acceder a la base de datos.

El funcionamiento será el siguiente:

1. La API recibe una solicitud de consulta.
2. Laravel genera la clave correspondiente al recurso solicitado.
3. Se consulta Redis.
4. Si la información existe, se devuelve directamente desde Redis. Este escenario se denomina **Cache Hit**.
5. Si la información no existe, se consulta MySQL. Este escenario se denomina **Cache Miss**.
6. El resultado obtenido desde MySQL se almacena temporalmente en Redis.
7. Finalmente, la API devuelve la información al consumidor externo.

Se seleccionó Cache-Aside porque permite mantener a Laravel como responsable del control de la caché y facilita la invalidación cuando un contacto es creado, actualizado o desactivado.

9.3 Cache Keys y TTL

Información	Cache Key	TTL
Detalle de un contacto	<code>crm:contacto:{id}</code>	300 segundos
Lista de contactos de un usuario	<code>crm:contactos:u:{userId}:{hashFiltros}:p:{page}</code>	60 segundos
Estadísticas del dashboard de un usuario	<code>crm:dashboard:u:{userId}</code>	120 segundos
Estadísticas generales del administrador	<code>crm:dashboard:admin</code>	120 segundos

El **TTL (Time To Live)** indica cuánto tiempo permanecerá almacenado un dato en Redis antes de eliminarse automáticamente.

Por ejemplo:

```
crm:contacto:15
TTL = 300 segundos
```

Esto significa que el detalle del contacto con identificador 15 podrá permanecer en caché durante un máximo de cinco minutos.

Los listados utilizan un TTL menor porque pueden cambiar con mayor frecuencia cuando se crean, actualizan o desactivan contactos.

9.4 Política de invalidación

La caché deberá invalidarse cuando se produzcan modificaciones sobre la información almacenada, para evitar que la API entregue datos desactualizados.

Creación de información

Cuando se crea un nuevo contacto mediante:

POST /contactos

no existe todavía una clave individual para ese recurso, pero sí pueden quedar desactualizados los listados y las estadísticas.

Por lo tanto, se invalidarán las claves relacionadas con:

crm:contactos:...

crm:dashboard:...

Después de esto, la siguiente consulta volverá a obtener los datos actualizados desde MySQL y los almacenará nuevamente en Redis.

Actualización de información

Cuando se actualiza un contacto mediante:

PUT /contactos/{id}

se eliminará la clave individual:

crm:contacto:{id}

También se invalidarán las listas y estadísticas relacionadas.

Por ejemplo:

DEL crm:contacto:15

De esta manera, la próxima solicitud de ese contacto producirá un Cache Miss y recuperará la versión actualizada desde MySQL.

Eliminación o desactivación de información

En el Módulo CRM los contactos no se eliminan físicamente de la base de datos, sino que se desactivan mediante el campo:

estado_activo = false

Cuando se ejecuta:

DELETE /contactos/{id}

se eliminará la clave:

crm:contacto:{id}

y también se invalidarán los listados y estadísticas relacionados.

Esto evita que Redis continúe devolviendo el contacto como activo cuando en MySQL ya fue desactivado.

- 9.5 [Flujo de Cache](#)
- 9.6 [Cache Hit](#)
- 9.7 [Cache Miss](#)
- 9.8 [Invalididez/Actualizacion de Cache](#)

10. Matriz de Trazabilidad

La matriz de trazabilidad relaciona las necesidades identificadas en el módulo CRM con las soluciones planteadas y los artefactos que las documentan. Permite comprobar que el análisis, la arquitectura, la API, la persistencia y la estrategia de caché responden a los objetivos del proyecto.

Necesidad / Requisito	Solución	Evidencia
Problemas de escalabilidad: atender el crecimiento de contactos y solicitudes.	Monolito modular con paginación, índices y posibilidad de desplegar varias instancias de la aplicación.	Diagrama 1: Link
Acoplamiento del legado: reducir la mezcla de presentación, lógica de negocio y acceso a datos.	Rediseño mediante MVC y separación propuesta en servicios, repositorios y DTO, con responsabilidades definidas.	Diagrama 4: Link
Exposición de funcionalidades: permitir que aplicaciones externas utilicen las operaciones del CRM.	Diseño de una API REST con endpoints, métodos HTTP, parámetros, DTO y respuestas documentadas.	Contrato OpenAPI: Link
Autenticación: controlar el acceso a las operaciones de la API.	Autenticación propuesta mediante JWT enviado como Bearer Token, acompañada de autorización por roles.	Contrato OpenAPI: Link
Arquitectura propuesta: comunicar la estructura del sistema y las responsabilidades de sus elementos.	Representación mediante el modelo C4 en los niveles de contexto, contenedores y componentes.	Diagramas 2, 3 y 4: Link
Persistencia: almacenar usuarios, contactos y sus clasificaciones	Modelo relacional en MySQL con las entidades Usuario, Contacto, Cliente, Proveedor y Empleado.	Diagrama 5: Link

Necesidad / Requisito	Solución	Evidencia
manteniendo relaciones consistentes.		
Creación de la base de datos: reproducir la estructura necesaria para el funcionamiento del CRM.	Script DDL con tablas, claves primarias y foráneas, restricciones e índices, acompañado de datos de prueba.	Script SQL: Link
Rendimiento: reducir consultas repetidas a MySQL y mejorar los tiempos de respuesta.	Estrategia propuesta con Redis y patrón Cache-Aside, utilizando claves, TTL e invalidación cuando cambian los datos.	Diagrama 6: Link

11. Validación de la solución

11.1 Validación del contrato API

Se realizó una validación estructural del contrato API del módulo CRM, revisando los siguientes aspectos:

Aspecto	Comprobación realizada
Validez del OpenAPI	Se analizó el archivo YAML y se validó mediante OpenAPIV30SpecValidator, sin errores. También se comprobó que sus 40 referencias internas apuntan a elementos existentes.
Visualización en Swagger UI	Queda pendiente comprobar la representación visual de las operaciones, parámetros, respuestas y esquemas, y obtener la captura correspondiente.
Endpoints	Se revisaron seis operaciones: inicio de sesión; listado, creación, consulta individual, actualización y desactivación de contactos. Cada operación tiene método HTTP, ruta, identificador y respuestas documentadas.
DTOs	Se revisaron 15 esquemas, incluyendo solicitudes de autenticación, creación y actualización de contactos, respuestas, paginación y errores. Se comprobaron sus tipos, campos obligatorios y referencias.
Códigos HTTP	Se verificó la documentación de los códigos 200, 201, 400, 401, 403, 404, 422 y 500, según las situaciones previstas para cada operación.

Aspecto	Comprobación realizada
Seguridad	Se comprobó que el esquema bearerAuth declara autenticación HTTP Bearer con formato JWT y se aplica a las cinco operaciones de contactos. El inicio de sesión no exige un token previo.

Esta validación corresponde a la estructura y documentación del contrato. Las respuestas reales del servidor y la aceptación o rechazo de tokens deberán comprobarse mediante pruebas funcionales cuando la API esté implementada.

Evidencia 2 – Validación Swagger: [Link](#)

11.2 Validación de arquitectura

La correspondencia entre los niveles C4 y la API permite comprobar que la arquitectura propuesta mantiene coherencia desde la visión general del CRM hasta la definición de sus operaciones.

Elemento	Correspondencia
C4 Nivel 1 – Contexto	Presenta al CRM como sistema principal e identifica a los usuarios y aplicaciones externas que interactúan con él. Estas interacciones determinan las funcionalidades que debe ofrecer el sistema.
C4 Nivel 2 – Contenedores	Amplía el CRM mostrado en el nivel anterior e identifica la aplicación Laravel, MySQL y Redis propuesto como caché. Define dónde se procesan las solicitudes y dónde se almacenan los datos.
C4 Nivel 3 – Componentes	Detalla la organización interna de la aplicación Laravel mediante controladores, servicios, repositorios, DTO y middleware propuestos. Distribuye las responsabilidades necesarias para atender las operaciones.
API – Contrato OpenAPI	Especifica las operaciones que exponen esos componentes: rutas, métodos HTTP, parámetros, DTO, respuestas y seguridad. Por ejemplo, las operaciones de contactos corresponden a los componentes responsables de su gestión.

Como ejemplo, la **consulta de un contacto** se relaciona de la siguiente manera:

- **Nivel 1:** una aplicación externa necesita consultar información del CRM.
- **Nivel 2:** la aplicación Laravel recibe la solicitud y accede a los datos necesarios.
- **Nivel 3:** el middleware verifica el acceso y los componentes de contactos procesan la consulta.

- **API:** el contrato define GET /contactos/{id}, el identificador requerido, la autenticación Bearer/JWT y las respuestas previstas.

La validación consiste en comprobar que los elementos detallados pertenecen al nivel anterior, que cada operación de la API tiene un componente responsable y que los DTO corresponden a la información del modelo de datos. Esta correspondencia documenta el diseño propuesto; su cumplimiento en el código se verifica durante la implementación.

11.3 Consistencia entre API, DTO, modelo de datos y base de datos

Se compararon las operaciones y esquemas del contrato OpenAPI con los modelos y las migraciones de Laravel. La correspondencia identificada es la siguiente:

API	DTO y datos involucrados	Modelo de datos	Base de datos
POST /auth/login	LoginRequest: correo y password.	Usuario.	El correo corresponde a usuarios.correo. La contraseña se verifica contra password_hash; no se almacena en texto plano.
POST /contactos y PUT /contactos/{id}	ContactoCreateRequest y ContactoUpdateRequest: nombre, apellido, correo, teléfono, dirección y empresa.	Contacto.	Los campos corresponden a columnas de contactos. Coinciden las longitudes máximas principales: nombre y apellido de 100 caracteres, correo de 160 y teléfono de 30.
Creación y actualización de clasificaciones	tipo_contacto: general, cliente, proveedor o empleado.	Contacto y sus entidades relacionadas.	contactos.tipo_contacto utiliza un ENUM con los mismos cuatro valores.
Contactos de tipo cliente	codigo_cliente, declarado como entero con mínimo 1.	Cliente relacionado con Contacto.	clientes.codigo_cliente es un entero sin signo y único. id_contacto funciona como clave primaria y foránea.

API	DTO y datos involucrados	Modelo de datos	Base de datos
Contactos de tipo proveedor o empleado	descripcion, puesto y departamento, según la clasificación.	Proveedor y Empleado.	Los campos se almacenan en proveedores y empleados, vinculados a contactos mediante id_contacto.
GET /contactos y GET /contactos/{id}	ContactoListResponse y ContactoResponse.	Contacto, Usuario y clasificaciones relacionadas.	Las respuestas representan información obtenida de las tablas relacionadas. Los elementos success, message y meta pertenecen al formato de respuesta de la API.
DELETE /contactos/{id}	ContactoEstadoResponse: identificador y estado_activo.	Estado lógico del Contacto.	Corresponde a actualizar contactos.estado_activo a falso, conservando el registro.

Las restricciones se complementan entre capas: los DTO describen los datos intercambiados, Laravel aplica las reglas de negocio y MySQL protege la integridad mediante claves, campos obligatorios y restricciones de unicidad.

Ajustes pendientes identificados:

- **Teléfono:** OpenAPI define entre 8 y 30 caracteres, mientras Laravel exige entre 8 y 15 dígitos y limita los caracteres permitidos. Esta regla debe precisarse en el contrato.
- **Clasificación:** los campos obligatorios según el tipo de contacto aparecen descritos en OpenAPI, pero su obligatoriedad condicional debe formalizarse, por ejemplo mediante esquemas oneOf.

La revisión demuestra correspondencia estructural entre los artefactos. La comprobación del flujo completo en ejecución requiere implementar y probar la API contra la base de datos.

11.4 Validación de caché

Se revisó la estrategia propuesta de **Cache-Aside**, comprobando su correspondencia con las operaciones de lectura y modificación del CRM. MySQL mantiene la información persistente y Redis almacena copias temporales para reducir consultas repetidas.

Operación	Comportamiento definido	Correspondencia con la estrategia
Consultar un contacto: GET /contactos/{id}	Consultar <code>crm:contacto:{id}</code> , con TTL de 300 segundos . Ante un Cache Miss, recuperar el contacto desde MySQL y almacenarlo temporalmente en Redis.	Permite reutilizar consultas del mismo contacto y reconstruir la caché cuando la entrada no existe o expira.
Listar contactos: GET /contactos	Utilizar <code>crm:contactos:u:{userId}:{hashFiltros}:p:{page}</code> , con TTL de 60 segundos .	Separa los resultados por usuario, filtros y página. El cálculo de <code>hashFiltros</code> debe incluir también el tamaño de página para distinguir respuestas diferentes.
Consultar resúmenes	Utilizar las claves de dashboard por usuario y administrador, con TTL de 120 segundos .	Reduce consultas repetidas de estadísticas y permite invalidarlas cuando cambian los contactos.
Crear un contacto: POST /contactos	Confirmar la inserción en MySQL e invalidar las listas y resúmenes afectados.	La siguiente consulta puede incorporar el nuevo contacto y actualizar los conteos.
Actualizar un contacto: PUT /contactos/{id}	Confirmar la modificación en MySQL, eliminar su clave de detalle e invalidar listas y resúmenes afectados.	Evita reutilizar la entrada anterior. Si cambia el propietario, deben invalidarse los resultados de ambos usuarios.

Operación	Comportamiento definido	Correspondencia con la estrategia
Desactivar un contacto: DELETE /contactos/{id}	Actualizar el estado lógico en MySQL y eliminar las entradas de caché afectadas.	Las consultas posteriores reflejan el nuevo estado, conservando el registro en la base de datos.

Para mantener esta correspondencia, la invalidación debe realizarse después de confirmar la transacción e incluir todas las variantes afectadas de filtros y paginación, además de los resúmenes administrativos. La autenticación y los permisos deben comprobarse también cuando existe un **Cache Hit**.

El TTL limita la duración de las copias almacenadas, mientras que la invalidación responde directamente a las modificaciones. No se almacenan contraseñas, hashes de contraseña ni tokens completos.

Esta revisión demuestra coherencia del diseño propuesto. Las pruebas de Cache Hit, Cache Miss, expiración e invalidación quedan pendientes de ejecución cuando Redis esté implementado.

Evidencia relacionada: Diagrama 6 y documento de estrategia de caché.

12. CONCLUSIONES

1. **El principal problema del sistema legado era el alto acoplamiento.** La presentación, las reglas de negocio y el acceso a MySQL compartían responsabilidades dentro de los mismos archivos. Esto favorecía la duplicación de código y dificultaba realizar modificaciones y pruebas sin afectar otras funcionalidades.
2. **El monolito modular propuesto ofrece una organización más adecuada para el CRM.** La separación mediante MVC, servicios, repositorios, DTO y middleware establece responsabilidades claras y facilita la exposición de una API REST. Además, permite mantener un único despliegue, con una complejidad operativa acorde con el alcance del proyecto.
3. **La propuesta busca mejorar mantenibilidad, escalabilidad, seguridad y disponibilidad.** La organización interna facilita el mantenimiento; la paginación, los índices y Redis contribuyen al rendimiento; JWT y la autorización por roles controlan el acceso; y el monitoreo, las copias de seguridad y la recuperación ante fallos

apoyan la disponibilidad. Estas mejoras requieren implementación y pruebas para comprobar sus resultados.

4. **Permanecen riesgos asociados al despliegue conjunto y a la consistencia de los datos.** Una falla crítica puede afectar toda la aplicación y los módulos no pueden escalarse de manera independiente. También existe riesgo de entregar información desactualizada si la invalidación de caché es incompleta, o de permitir accesos indebidos si los controles de autorización se aplican incorrectamente.
5. **Antes de producción deben completarse y verificarse los componentes propuestos.** Es necesario implementar la API, JWT y Redis, resolver las diferencias entre DTO y validaciones de Laravel, y ejecutar pruebas funcionales, de integración, seguridad y carga. También deben configurarse HTTPS, credenciales mediante variables de entorno, monitoreo, respaldo y restauración de datos, y un procedimiento de reversión del despliegue.

13.REFERENCIAS

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.^a ed.). Addison-Wesley Professional. [Enlace editorial](#).

Brown, S. (s. f.). *The C4 model for visualising software architecture*. <https://c4model.com/>.

International Organization for Standardization & International Electrotechnical Commission. (2023). *Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model* (ISO/IEC 25010:2023). [Ficha oficial del estándar](#).

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). RFC Editor. <https://www.rfc-editor.org/rfc/rfc7519>.

Laravel. (s. f.). *Laravel 12.x documentation*. <https://laravel.com/docs/12.x>.

Microsoft. (s. f.). *Cache-Aside pattern*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside>.

OpenAPI Initiative. (2020, 20 de febrero). *OpenAPI specification* (versión 3.0.3). <https://spec.openapis.org/oas/v3.0.3.html>.

Oracle. (s. f.). *MySQL 8.4 reference manual*. <https://dev.mysql.com/doc/refman/8.4/en/>.

Redis. (s. f.). *Redis documentation*. <https://redis.io/docs/latest/>.

Richards, M., & Ford, N. (2025). *Fundamentals of software architecture* (2.^a ed.). O'Reilly Media. [Enlace editorial](#).

SmartBear. (s. f.). *Swagger UI*. <https://swagger.io/open-source/swagger-ui/>.